

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Фронт-энд разработка

Отчет

по лабораторной работе  
«Интеграция фронтенд-приложения с tock API»

Выполнил:

Баранов Владимир

Группа

J3213

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

## **Задача**

Цель лабораторной работы заключалась в расширении ранее сверстанного фронтенд-приложения и переводе его от статического сценария к работе с внешним API. В рамках задания требовалось подключить страницы проекта к mock REST API, организовать регистрацию и вход пользователя, ограничить доступ к персональным данным средствами авторизации, а также реализовать базовые сценарии работы с ключевыми сущностями предметной области: экспериментами, моделями, метриками и артефактами.

## **Ход работы**

Работа выполнялась поэтапно: сначала было подготовлено окружение и моковый API, затем реализован клиентский слой взаимодействия с сервером, после чего были подключены формы авторизации и основные страницы проекта. На завершающем этапе был проведен рефакторинг JavaScript-кода и проверена корректность работы основных пользовательских сценариев.

### **1. Подготовка окружения и mock API**

Для имитации серверной части был создан файл `package.json` и подключены зависимости `axios`, `json-server` и `json-server-auth`. Такой набор позволил организовать тестовую клиент-серверную связку без развертывания полноценного backend-приложения.

В проект были добавлены npm-скрипты для запуска mock API на локальном адресе `http://localhost:3000` и для обслуживания статических файлов фронтенда. Отдельно был подготовлен файл `api/db.json`, в котором описаны основные коллекции данных: пользователи, эксперименты, модели, метрики и артефакты.

Использование `json-server-auth` позволило смоделировать защищенные маршруты и приблизить учебный проект к реальному сценарию работы с аутентификацией и приватными ресурсами.

### **2. Реализация клиентского слоя доступа к API**

Для работы с HTTP-запросами был создан модуль `js/api.js`. В нем реализована единая точка создания `axios`-клиента с базовым URL и автоматическим добавлением заголовка `Authorization` при наличии токена доступа.

Дополнительно были выделены функции для сохранения и чтения данных сессии из `localStorage`. Это обеспечило централизованную работу с `accessToken` и информацией о текущем пользователе, а также упростило повторное использование логики на разных страницах приложения.

В модуле `js/auth.js` были реализованы функции регистрации и входа пользователя, проверка наличия авторизации на приватных страницах и динамическое изменение элементов навигации в зависимости от состояния сессии.

### **3. Подключение авторизации и защита пользовательских сценариев**

На странице регистрации была настроена отправка пользовательских данных в API, добавлена фронтенд-проверка совпадения паролей и реализовано автоматическое сохранение сессии после успешного создания учетной записи.

На странице входа реализован запрос к API для аутентификации пользователя с последующим сохранением `accessToken` и данных профиля. После успешного входа пользователь перенаправляется в личный кабинет. Для страниц `dashboard.html`, `experiments.html`, `experiment.html` и `models.html` была добавлена проверка авторизации. При отсутствии токена доступ к приватным данным блокируется, а пользователь перенаправляется на страницу входа. Таким образом, персональные данные и действия над сущностями стали доступны только авторизованному пользователю.

### **4. Интеграция основных страниц проекта с API**

Личный кабинет пользователя был переведен на динамическую подгрузку данных. На странице `dashboard` отображаются агрегированные показатели: количество экспериментов, моделей и артефактов, а также

статистика по статусам экспериментов Running, Completed и Failed. Кроме того, реализован вывод последних экспериментов на основе данных, полученных из API.

Страница experiments была связана с API для получения списка экспериментов конкретного пользователя. Были реализованы фильтры по дате, метрике, статусу и тегам. Через форму создания можно добавить новый эксперимент с указанием названия, этапа, описания, целевой метрики, тегов и связи с моделью. Для уже существующих экспериментов доступны операции Start, Finish, Fail через PATCH-запросы и удаление через DELETE.

На детальной странице experiment реализована загрузка эксперимента по идентификатору из URL, отображение основных атрибутов и интерактивное управление содержимым. Пользователь может менять статус эксперимента, добавлять заметки, создавать новые метрики и прикреплять артефакты. После выполнения операций списки логов, метрик и артефактов обновляются без перезагрузки страницы.

Страница models также была переведена на работу с серверными данными. Реализована загрузка моделей пользователя, форма регистрации новой модели и действия над существующими записями: перевод модели в статус продакшн, архивирование и удаление.

## **5. Рефакторинг и проверка качества решения**

После завершения интеграции был проведен рефакторинг JavaScript-кода. Устаревшие объявления var были заменены на const и let, повторяющиеся фрагменты логики вынесены в переиспользуемые функции, а код модуля experiments.js был упрощен и структурирован.

Дополнительно была проверена согласованность структуры данных в db.json, корректность обработки ответов API и стабильность основных сценариев: регистрация, вход, просмотр личного кабинета, создание эксперимента, изменение статусов, работа с моделями, метриками и артефактами.

## **Вывод**

В ходе выполнения лабораторной работы статический фронтенд-проект был успешно преобразован в упрощенное клиент-серверное приложение, взаимодействующее с внешним mock API. Были реализованы регистрация и вход пользователя, хранение сессии, защита приватных страниц, а также динамическая работа с основными сущностями предметной области: экспериментами, моделями, метриками и артефактами.

Полученный результат показывает, что учебный проект больше не является набором изолированных HTML-страниц: данные загружаются и изменяются через HTTP-запросы, пользовательские сценарии опираются на авторизацию, а ключевые действия выполняются в формате CRUD-операций и workflow-статусов.

Таким образом, поставленная цель достигнута. Лабораторная работа продемонстрировала практическое применение темы «Имитация работы с API» и показала, каким образом фронтенд-приложение может быть поэтапно переведено к модели работы с внешним сервисом без использования полноценного backend на раннем этапе разработки.